

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 5: Eloquent ORM dan Relationship

Pemetaan Objek ke Tabel, Relationship Antar Model, dan Query yang Efisien

Yang Akan Kamu Pelajari

Model Eloquent bisa melakukan jauh lebih banyak daripada sekadar mengambil dan menyimpan data satu per satu. Pertemuan ini membahas:

1. Apa itu **ORM**, dan bagaimana Eloquent menebak nama tabel serta menjalankan operasi dasar (ambil, buat, ubah, hapus) tanpa SQL manual
2. Cara menyambungkan model lewat **relationship** (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`)
3. Kenapa relationship yang dimuat sembarangan bisa diam-diam menjalankan puluhan query (**N+1**), dan cara menghindarinya lewat **eager loading**
4. Cara menampilkan data ribuan baris tanpa membebani satu halaman penuh, lewat **pagination**

Bagian 1

Konsep ORM: Memetakan Class ke Tabel

Kenapa `Article::find(1)` bisa langsung tahu harus mengambil dari tabel `articles`, padahal kamu tidak pernah menulis nama tabel itu di mana pun? Jawabannya adalah ORM.

ORM (Object-Relational Mapping): teknik yang memetakan class dalam kode ke tabel di basis data, dan objek ke baris, sehingga operasi basis data ditulis lewat method dan properti object, bukan query SQL mentah.

Class (Model)

→

ORM

→

Tabel (baris)

Eloquent memakai pola **Active Record**: satu class model merepresentasikan satu tabel, sekaligus menyediakan method untuk query, simpan, dan hapus datanya sendiri, bukan class terpisah untuk tiap tugas.

Model Eloquent: Satu Class per Tabel

```
class Article extends Model
{
    //
}
```

Isinya kosong, tapi jangan salah: class ini sudah bisa mengambil, menyimpan, mengubah, dan menghapus data tanpa satu baris SQL pun ditulis. Semua kemampuan itu datang gratis dari satu baris `extends Illuminate\Database\Eloquent\Model`.

- Satu class model mewakili satu tabel; satu object hasil query mewakili satu baris di tabel itu
- Model "kosong" begini normal, sebagian besar perilakunya sudah otomatis lewat konvensi (slide berikutnya)

Konvensi Eloquent: Tebakan Otomatis, Bukan Kebetulan

Nama tabel jamak snake_case yang selama ini kamu tulis di migrasi bukan kebetulan. Eloquent menebak tiga hal dari nama class model itu sendiri, tanpa kamu konfigurasi apa pun.

Model	Tabel (konvensi)	Primary Key	Timestamps
Article	articles	id (otomatis)	created_at / updated_at (otomatis)
Category	categories	id (otomatis)	created_at / updated_at (otomatis)

Prosesnya: ambil nama class, ubah ke snake_case, lalu jadikan jamak. `Category` jadi `categories`. Nanti di jobsheet kamu akan membuat model `TransactionDetail`, dan dengan aturan yang sama Eloquent akan mencarinya di tabel `transaction_details`, tanpa kamu bilang apa-apa.

Mengambil Data: Query Dasar Eloquent

```
$articles = Article::all();  
$article = Article::find(1);  
$articles = Article::where('title', 'like', '%laravel%')->get();  
$article = Article::where('title', 'like', '%laravel%')->first();
```

Empat baris, dua bentuk hasil yang berbeda. `all()` dan `get()` mengembalikan **Collection**, bisa berisi banyak object model sekaligus. `find()` dan `first()` mengembalikan **satu** object model saja, atau `null` kalau memang tidak ketemu, jadi selalu siapkan penanganan untuk kemungkinan `null`-nya.

Membuat Data: `create()` vs `new` + `save()`

Ada dua cara menulis baris baru, dan keduanya berakhir persis sama di database.

Lewat `create()`

```
Article::create([
    'title' => 'Judul Baru',
    'body' => 'Isi artikel...',
]);
```

Lewat `new` + `save()`

```
$article = new Article();
$article->title = 'Judul Baru';
$article->body = 'Isi artikel...';
$article->save();
```

`create()` lebih ringkas dan lebih umum dipakai, tapi ada satu jebakan yang langsung muncul begitu kamu mencobanya: Eloquent bisa menolaknya mentah-mentah.

Mass Assignment dan `$fillable`

Coba `Article::create([...])` pada model baru buatan `make:model`, dan Eloquent melempar `MassAssignmentException`. Ini bukan bug, ini pengaman.

Mass Assignment: mengisi banyak kolom model sekaligus lewat satu array, seperti pada `create()`. Eloquent memblokirnya secara default lewat `MassAssignmentException`, kecuali kolom mana yang boleh diisi massal sudah didaftarkan lebih dulu.

```
class Article extends Model
{
    protected $fillable = ['title', 'body'];
}
```

`$fillable` adalah daftar putih kolom yang boleh diisi massal. Tanpa itu, Eloquent menolak `create()` / `update()` massal secara default, supaya kolom sensitif seperti peran atau status tidak bisa diisi diam-diam lewat input yang tidak terduga.

Mengubah dan Menghapus Data

```
$article = Article::find(1);  
$article->title = 'Judul Diperbarui';  
$article->save();  
  
// atau langsung lewat satu pemanggilan:  
Article::find(1)->update(['title' => 'Judul Diperbarui']);  
  
Article::find(1)->delete();
```

Pola yang sama seperti membuat data: ubah properti lalu `save()`, atau langsung lewat satu pemanggilan `update()`. `delete()` tidak punya versi "batal", jadi biasanya `find()` dan cek dulu sebelum memanggilnya di kode sungguhan.

Method lengkap: dokumentasi resmi Laravel Eloquent (`firstOrCreate` , `updateOrCreate` , dan lainnya)

Bagian 2

Tanpa Relationship vs Dengan Relationship Eloquent

Tanpa relationship Eloquent

- Setiap butuh data terkait, tulis ulang query join secara manual
- Rawan salah ketik nama kolom join, berulang di banyak tempat

Dengan relationship Eloquent

- Dideklarasikan sekali di model
- Dipakai di mana saja lewat properti atau method, tanpa menulis ulang join

Mirip pohon silsilah keluarga: garis keturunan digambar sekali, lalu siapa pun yang membaca pohon itu langsung tahu hubungannya, tanpa menelusuri dokumen satu per satu.

Relationship: Menghubungkan Model

Relationship: metode pada model Eloquent yang mendeskripsikan bagaimana satu tabel terhubung ke tabel lain, dipakai layaknya properti biasa, meskipun sebenarnya menjalankan query di belakang layar.

```
class Author extends Model
{
    public function articles(): HasMany
    {
        return $this->hasMany(Article::class);
    }
}

class Article extends Model
{
    public function author(): BelongsTo
    {
        return $this->belongsTo(Author::class);
    }
}
```

hasOne: Relationship Satu ke Satu

Ganti `hasMany` dengan `hasOne`, dan Eloquent langsung tahu untuk berhenti di baris pertama saja:

```
class Author extends Model
{
    public function profile(): HasOne
    {
        return $this->hasOne(Profile::class);
    }
}
```

Satu penulis punya satu profil, bukan beberapa, jadi `$author->profile` langsung mengembalikan satu object, bukan Collection.

belongsToMany: Relationship Banyak ke Banyak

```
class Article extends Model
{
    public function tags(): BelongsToMany
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

Satu artikel bisa punya banyak tag, dan satu tag bisa dipakai di banyak artikel, jadi tidak ada satu pun kolom foreign key yang cukup. Perlu tabel pihak ketiga di tengah, `article_tag`, mengikuti konvensi nama: kedua nama model tunggal,urut abjad, dipisah garis bawah.

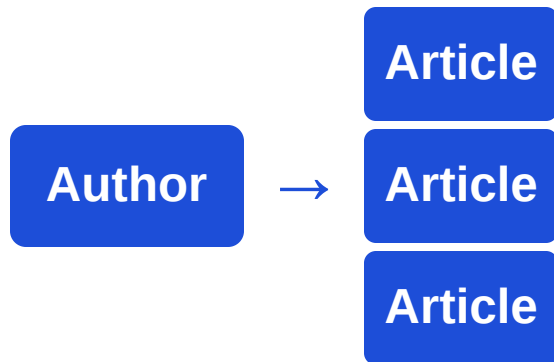
Bentuk Relationship: Empat Pola Dasar

Empat method yang baru saja kamu lihat sebenarnya cuma dua ide: siapa punya satu, dan siapa punya banyak, ditambah arah baliknya.

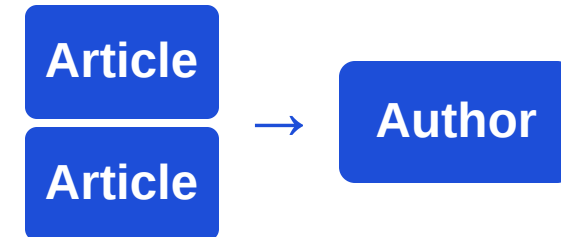
hasOne: satu induk, satu anak



hasMany: satu induk, banyak anak



belongsTo: arah balik dari hasMany



belongsToMany: banyak ketemu banyak



Empat Jenis Relationship: Ringkasan

Relationship	Arti	Contoh
hasOne	Satu baris induk punya satu baris terkait	Author punya satu Profile
hasMany	Satu baris induk punya banyak baris terkait	Author punya banyak Article
belongsTo	Baris ini menunjuk balik ke satu baris induk	Article menunjuk ke satu Author
belongsToMany	Banyak ke banyak lewat pivot table	Article dan Tag saling terhubung

Nama Metode Relationship Bukan Sekadar Kosmetik

Tulis `articles()` sekali di model `Author`, dan nama itu langsung berlaku di tiga tempat berbeda:

- **Pemanggilan metode** `$author->articles()` : mengembalikan query builder, bisa dirangkai dengan `->where()`, `->orderBy()`, dst.
- **Akses properti** `$author->articles` : langsung mengembalikan koleksi hasilnya, tanpa tanda kurung
- **String di dalam** `with('articles')` : memberi tahu Eloquent relationship mana yang harus dimuat lebih awal, topik slide berikutnya

Bagian 3

Masalah N+1

Relationship yang gampang dipakai justru bisa diam-diam jadi masalah performa paling umum di aplikasi Laravel mana pun, dan namanya cukup aneh: N+1.

Lazy Loading: relationship baru dimuat saat properti itu benar-benar diakses, baris demi baris, sumber utama masalah N+1.

N+1: pola query bermasalah, satu query untuk daftar utama, ditambah satu query terpisah untuk setiap baris guna memuat relationship-nya.

Menampilkan 15 artikel beserta nama penulisnya, tanpa eager loading: 1 query daftar + 15 query penulis = **16 query**, hanya untuk satu halaman berisi 15 baris.

Pendekatan	Jumlah Query
Lazy loading (tanpa <code>with()</code>)	1 query daftar + N query relationship (N+1)
Eager loading (<code>with('author')</code>)	2 query total

N+1 vs Eager Loading: Perbandingan Visual

Lazy loading: 16 tembakan query terpisah

1 query: daftar artikel

query author #1

query author #2

query author #3

... 12 lagi

Eager loading: 2 tembakan, selesai

1 query: daftar artikel

1 query: semua author sekaligus

Eager Loading

Eager Loading: teknik memuat relationship di muka lewat `with()` , dalam jumlah query yang tetap kecil, alih-alih memuatnya satu per satu saat baru dibutuhkan.

```
$articles = Article::with('author')  
    ->latest()  
    ->paginate(15);
```

Satu kata tambahan, `with('author')` , dan 16 query tadi langsung menyusut jadi 2. Jumlahnya juga tidak bergantung pada berapa banyak baris ditampilkan, hanya pada berapa lapis relationship yang dimuat, jadi 15 artikel atau 150 artikel sama-sama 2 query total, bukan 2 dikali jumlah baris.

Eager Loading Berjenjang: Notasi Titik

Relasinya bisa lebih dari satu lapis. Tulis lewat notasi titik, dan Eloquent tetap memuat semuanya dalam jumlah query yang tetap:

```
$authors = Author::with('articles.comments')->get();
```

Author



Article



Comment

Satu baris ini memuat setiap author, relationship `articles`-nya, **dan** relationship `comments` di dalam tiap article sekaligus, dua lapis relationship dalam satu notasi titik, tanpa jumlah query ikut bertambah seiring jumlah barisnya.

Bagian 4

Konsep Dasar Pagination: LIMIT dan OFFSET

Menampilkan 2.500 baris sekaligus di satu halaman jelas bukan ide bagus, untuk pengguna maupun untuk database-nya. Solusinya sebenarnya sudah ada di level SQL, jauh sebelum Laravel ikut campur.

LIMIT dan OFFSET: klausa SQL yang membatasi jumlah baris dikembalikan (LIMIT) dan melompati sejumlah baris di awal (OFFSET), dasar dari semua teknik pagination, di framework apa pun.

```
-- Halaman 2, 10 baris per halaman  
SELECT * FROM articles  
ORDER BY title  
LIMIT 10 OFFSET 10;
```

baris 1-10 (halaman 1)

baris 11-20 (halaman 2)

baris 21-30 (halaman 3)

OFFSET dihitung dari nomor halaman lewat rumus $(\text{halaman} - 1) * \text{jumlah_per_halaman}$: halaman 1 jadi OFFSET 0, halaman 2 jadi OFFSET 10, halaman 3 jadi OFFSET 20, dan seterusnya.

Tanpa Laravel: Menghitung Semuanya Manual

Begini rupanya kalau LIMIT dan OFFSET tadi ditulis langsung di controller, tanpa bantuan `paginate()` :

```
$perPage = 10;
$page = request('page', 1);
$offset = ($page - 1) * $perPage;

$articles = Article::orderBy('title')
    ->skip($offset)
    ->take($perPage)
    ->get();

$total = Article::count();
$lastPage = (int) ceil($total / $perPage);
```

Enam baris, dan itu baru menghitung halamannya saja. Navigasi ("halaman 1 2 3 ...") masih harus ditulis sendiri di view, lengkap dengan link ke tiap nomor halaman. Ini pekerjaan yang sama persis di hampir setiap listing di aplikasi mana pun, jadi wajar kalau Laravel akhirnya membungkusnya jadi satu method saja.

`paginate()` : Satu Method Menggantikan Semuanya

Pekerjaan	Tanpa Laravel (manual)	Dengan <code>paginate()</code>
Hitung offset dari nomor halaman	Ditulis manual	Otomatis, baca dari <code>?page=</code>
Hitung total baris & jumlah halaman	Query <code>COUNT()</code> terpisah	Otomatis, sudah termasuk
Navigasi halaman (link nomor)	Ditulis manual di view	<code>{{ \$articles->links() }}</code>

Dibaliknya, `paginate()` tetap menjalankan LIMIT dan OFFSET yang sama seperti slide sebelumnya, ditambah satu query `COUNT()`. Laravel hanya membungkus mekanisme itu supaya tidak perlu ditulis ulang setiap kali.

Menerapkan `paginate()` di Controller

Enam baris tadi sekarang tinggal satu pemanggilan method, lengkap dengan metadata halaman siap pakai:

```
public function index()
{
    $articles = Article::with('author')
        ->orderBy('title')
        ->paginate(10);

    return view('articles.index', compact('articles'));
}
```

```
<!-- resources/views/articles/index.blade.php -->
<div class="mt-4">
    {{ $articles->links() }}
</div>
```

Satu baris `{{ $articles->links() }}` di view sudah cukup, tidak ada berkas navigasi tambahan yang perlu dibuat manual.

Kapan Pakai Pagination

Pakai `paginate()` setiap kali jumlah baris berpotensi tumbuh tak terbatas seiring waktu, seperti daftar artikel atau riwayat komentar. `get()` biasa masih cukup untuk daftar yang memang kecil dan tetap kecil, seperti daftar tag pada dropdown filter.

Membuktikan Pagination Bekerja

Kode di atas terlihat benar, tapi bagaimana memastikan halaman 2 sungguhan menampilkan baris yang berbeda dari halaman 1, bukan diam-diam mengulang data yang sama?

```
php artisan tinker
>>> Article::orderBy('title')->paginate(10)->pluck('id');
>>> Article::orderBy('title')->paginate(10, ['*'], 'page', 2)->pluck('id');
```

Bandingkan hasil kedua perintah. ID yang tumpang tindih antara halaman 1 dan halaman 2 berarti ada masalah pada `orderBy` atau datanya, bukan pada `paginate()` - nya.

Menerapkan pada Simple POS

Konsep relationship, eager loading, dan pagination ini akan kamu terapkan langsung pada studi kasus Simple POS di jobsheet praktikum: memetakan relationship

`Category - Product` dan `Transaction - TransactionDetail - Product`, menghindari N+1 pada listing 2.500 transaksi, dan membuktikan pagination pada 300 produk.

Kode `Product::take(12)->get()` yang sudah kamu tulis di Pertemuan 4 sekarang punya penjelasan penuh: itu query dasar Eloquent yang dibahas di awal pertemuan ini.

Kode lengkap: github.com/se-polinema/simple-pos-ch05

Rangkuman

- ORM seperti Eloquent (pola Active Record) memetakan class Model ke tabel lewat konvensi otomatis, dengan operasi CRUD dasar dan `$fillable` untuk mencegah mass assignment
- Relationship (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`) dideklarasikan sekali di model, lalu dipakai lewat pemanggilan metode, akses properti, atau string di dalam `with()`
- Lazy loading memicu satu query tambahan per baris per relationship (masalah N+1); eager loading, termasuk notasi titik berjenjang, memuat relationship dalam jumlah query tetap, tidak bergantung jumlah baris
- `paginate()` membagi listing besar jadi halaman-halaman kecil; halaman berbeda bisa dibuktikan benar-benar berbeda dengan membandingkan ID hasil antar halaman

Referensi & Diskusi

Dokumentasi resmi Laravel (Eloquent, Relationships, Pagination)

Kode lengkap: `github.com/se-polinema/simple-pos`

Pertemuan berikutnya: Validasi & Keamanan Input